

量子コンピューティングI NISQ編 #2

第6回 QAOAアルゴリズム

情報工学科 佐藤貴彦

#1	#2	#3	#4	#5	#6	#7	#8	#9	#10	#11	#12	#13	#14
Intro.	量子回路	Deutsch	Grover	実機実装	本日	量子AI	誤り訂正	論理演算	符号理論	量子通信	アプリ	インフラ	総括
基礎編 #2-4 (完)				NISQ 編 #5-7			FTQC 編 #8-10			量子インターネット編 #11-14			

この回の到達点

- ① NISQ アルゴリズムのハイブリッド設計を説明できる
- ② QAOA の構造を理解する (Cost / Mixer / Ansatz)
- ③ ライツアウトを QAOA で解決し、Grover と CX 数で比較できる

NISQ で何ができるのか

Grover はシミュレータでは高確率で解を得たが、ノイズを考慮すると成功確率を大きく落とした

第5回 NISQ#1 で見たこと

- ・ 基礎編で組んだ Grover (理想シミュレータ) は高確率で正解を出した
- ・ 第5回で同じ Grover を FakeBrisbane (実機相当ノイズ) で動かすと成功確率が大きく低下した
- ・ 原因: **CX 数 × ゲートエラー率の累積** で解の振幅が指数的に減衰してしまう
- ・ エラー訂正できないNISQには大きな制約がある (測定・ゲートのエラー抑制手法も学んだ)

本回の問い

- ・ 実機ノイズの制約のもとで、**どんなアルゴリズムならNISQ で動くのか?**
- ・ 本日は **QAOA** を中心に、NISQアルゴリズムを学ぶ
- ・ Grover と QAOA を CX 数 + 量子回路実行回数で**定量比較** し、NISQ アルゴリズムのトレードオフ構造を掴む

まずは「NISQ アルゴリズムの思想」を見てみましょう : Sec. 1 ~

Sec. 1

NISQ アルゴリズムの思想

エラー訂正できない → 短い回路 → ハイブリッド設計 を 10 枚 で辿る
(Sec.1 合計 10 枚 / 約 23.5 分)

1

2

3

エラー訂正できないので、回路を短くする

少なくとも、量子ビットの寿命 (T_2) 内に終わる depth に収まらないと、計算に失敗する

ゲートによるノイズ蓄積の構造

- ・ 1CNOT ゲートあたリエラー率 p (現代の実機だと0.1%~0.5%) で、 N ゲートを通せば 忠実度 $\approx (1-p)^N$
- ・ 例: $p = 0.005$ 、 $N = 200 \rightarrow$ 忠実度 $\approx 37\%$ 、 $N = 500 \rightarrow$ 忠実度 $\approx 8\%$ 深い回路ほど情報が壊れていく

誤り訂正が可能な場合(FTQC)

- ・ 1 論理 qubit に 100~1000 物理 qubit を割く
- ・ エラー訂正符号で論理エラー率を抑え込む
- ・ 実用化は近未来 (#8~#10 FTQC 編で扱う)

将来的にはこちらでGroverも動くはず

NISQ の場合

- ・ 誤り訂正なしで、depth が T_2 コヒーレンス時間内に収まる範囲で計算する
- ・ ゲート自体もノイズを生み出すことを考慮する

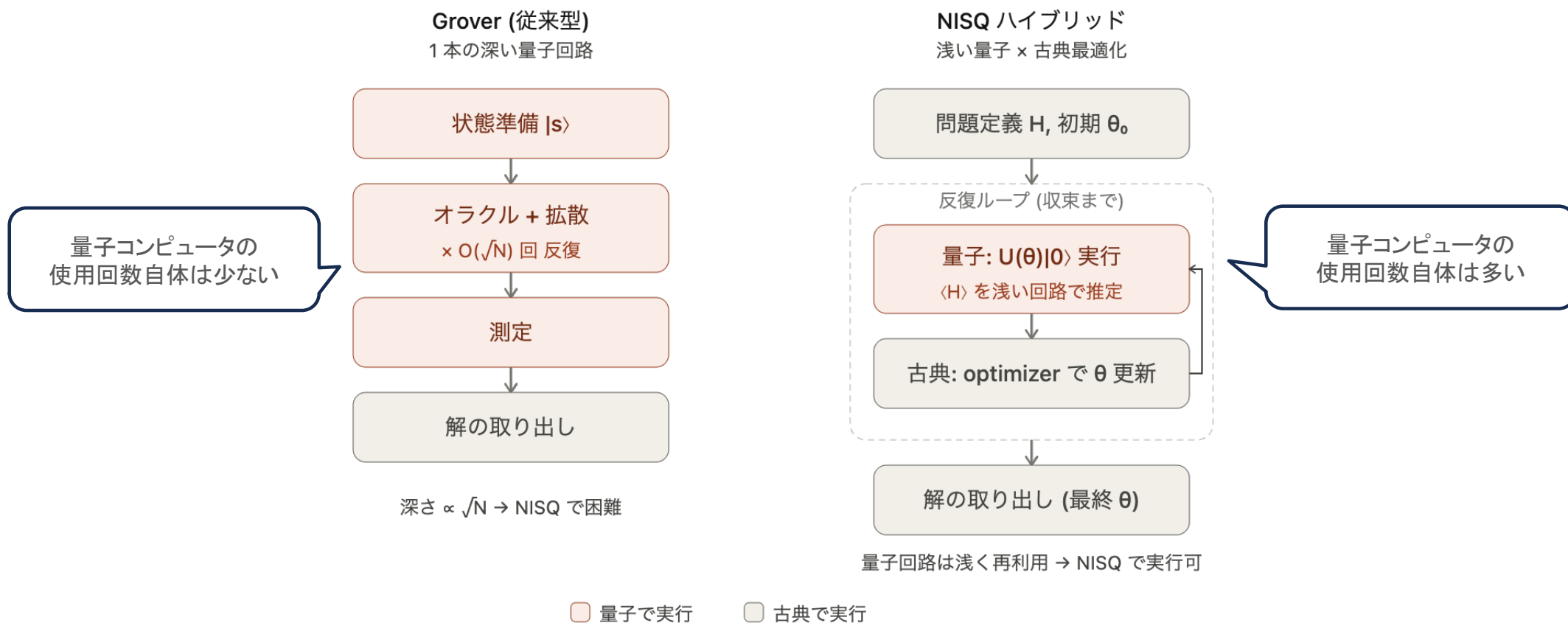
本日学ぶ QAOA はこの思想に基づくが、何が解ける？

※ T_2 = qubit の量子状態が壊れるまでの典型時間 (μs オーダー)、コヒーレンス時間とも呼ぶ

※ 誤り訂正と NISQ 時代の境界は副読本シリーズで詳述予定

古典に追従困難な部分だけ、量子に肩代わりさせる

計算全部を量子でやる必要はない：短い回路に収めるため、古典追従困難な部分だけを量子に切り出す



パラメータ更新は古典 optimizer に任せて回路を短く保つ

量子は「コスト関数 1 回分の値」だけ評価 ↔ 古典が関数パラメータを更新 をループさせる

量子 (QAOAの場合)

- ・パラメータ付き回路を実行
(パラメータ = β, γ)
- ・コスト関数の値 $\langle H_C(\beta, \gamma) \rangle$ を返す
- ・”探索”は行わず、
回路 depth を浅く保つ

$\langle H_C \rangle$ の値



次の (β, γ)



古典

- ・ (COBYLA 等の) optimizer を用いて
 $\langle H_C \rangle$ から (β, γ) を最適化する
- ・最適化したパラメータを
次の β, γ として量子側に返して
再実行してもらう
- ・解が収束するまでループを続ける

注: (β, γ) = QAOA の Cost layer / Mixer layer のパラメータ (詳細 Slide 18)、
 $\langle H_C \rangle$ = コスト Hamiltonian の期待値 (詳細 Slide 21)

古典アルゴリズムによる最適化

(松谷先生の)アルゴリズム論IIの
第12-13回をお楽しみに！

古典最適化の構造を 2 つのパートに分けて見る

① コスト関数の評価

- ・ パラメータ β, γ を入力
- ・ コスト関数の値 $f(\beta, \gamma)$ を出力

計算量の重さ:

- ・ 問題によっては高次元の積分や指数和を計算する
- ・ 古典では指数時間の計算になることがある

具体例:

- ・ MaxCut: グラフ全 2^n 解の平均カット数
- ・ QAOA cost: $\langle \psi(\beta, \gamma) | H_C | \psi(\beta, \gamma) \rangle$

② optimizer

- ・ コスト関数の値 $f(\beta, \gamma)$ を入力
- ・ 次に試すべき (β, γ) を出力

計算量の重さ:

- ・ パラメータ次元に対して多項式時間で計算される
- ・ 古典で十分速い

具体例:

- ・ COBYLA / Nelder-Mead / SPSA
- ・ SciPy.optimize.minimize で標準的に提供されている

①の評価が重く②の更新は軽い、という非対称性がある。この非対称性を量子で活用する。

注: H (Hamiltonian) = この文脈では「コスト関数を Z や X の演算子で書き直したもの」(物理的意味は副読本)

古典・量子ハイブリッドアルゴリズムによる最適化

Slide 7 の 2パートのうち、①だけを量子（NISQ）に置き換える

① NISQによるコスト関数の評価

コスト関数を 量子状態の期待値 として書き直す:

$$f(\beta, \gamma) \Leftrightarrow \langle \psi(\beta, \gamma) | H_C | \psi(\beta, \gamma) \rangle$$

- ・ H_C はコスト関数の対応する Hamiltonian
- ・ $|\psi(\beta, \gamma)\rangle$ はパラメータ付き回路の出力状態

なぜ量子なのか:

- ・ 古典で指数時間の高次元期待値計算を、
量子回路 1 個のサンプリングで近似する

② Optimizerは古典のまま

- ・ COBYLA / Nelder-Mead / SPSA 等をそのまま使う
- ・ 量子側から $\langle H_C \rangle$ の値が返ってきたら、古典最適化と同じ手順で次の (β, γ) を提案

なぜ量子化しないのか:

- ・ optimizer は古典で十分速い (多項式時間)
- ・ 量子回路自体は短く保ち、適材適所でループを回す

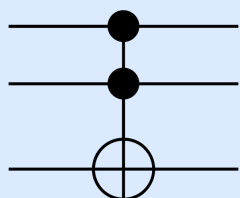
NISQ ハイブリッドの構造: 量子で（古典で重い）関数評価 + 古典で軽い更新 + 反復で最適 (β, γ) を発見
Sec.2 以降で、より具体的なアルゴリズムとして QAOA を見ていきましょう

考察 | Toffoliゲートの実装コスト

ポストNISQ (FTQC) アルゴリズムで多用されるToffoliゲートの物理実装を考えてみよう

実装 1 抽象回路

Toffoli (CCX) ゲート

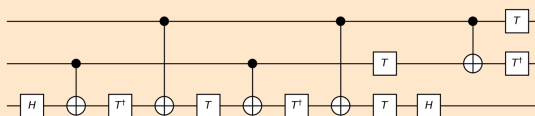


- 3 qubit の論理 AND ゲート
- GroverのDiffusionや量子AIのQRAMデータ構造など FTQC では活躍の場が多い

直接実装可能な中性原子系もある

実装 2 標準分解

2-qubit + 単 qubitゲートに分解



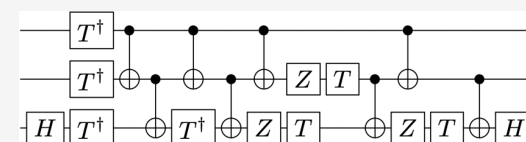
CX 6 個

- CX (CNOT) ゲート 6個と、単一 qubit ゲート (T, T[†], H 等) の組み合わせで等価実装が可能
- Qubit同士は全結合が必要

イオントラップ系で直接実装可能

実装 3 接続性制約下での分解

1次元接続での実装



CX 8 個

- 実機の qubit 接続グラフ (例: heavy-hex) では離れた qubit 同士に直接 CX が実行できない
- 特殊な分解やSWAP (= 3 CX) の挿入によるオーバーヘッドが避けられない

Toffoliゲート 1 個でも物理実装のコストは意外と重い。

NISQ ではCXのエラーが重要なため、**Toffoli を多用するアルゴリズムの実装は厳しい**

補足 | NISQ のゲート事情は系ごとに異なる

直接物理実装できるゲート（nativeゲート）は物理系によって異なる。以下に典型例をまとめる。

系	2 qubit gate	Toffoli の扱い	代表的なチーム
超伝導	CX (CZ) が native	CX 8 個 + 単 qubit ゲート群 (オーバーヘッドあり)	IBM ・ Google ・ Rigetti ・ 理研
イオントラップ	XX (Mølmer-Sørensen) が native	全結合のため 教科書通りの CX 6個で実装できる	Quantinuum ・ IonQ
中性原子	CCZ も含む多体ゲートが Rydberg ブロッケードで native	Toffoli を直接実装可能 (分解不要)	QuEra ・ Atom Computing ・ Pasqal

含意: 同じアルゴリズムでも、実装基盤によって回路の depth と CX 数が大きく変わる。
本日 Slide 40 の比較表でも transpile + 分解後の CX 数を 1 つの基盤 (超伝導 / Heron 想定) で計測する。

NISQ ハードウェアの多様性詳細は副読本シリーズで詳述予定

NISQ アルゴリズムを俯瞰する

初期のNISQアルゴリズムはVQE・QAOA・QMLの「コスト関数評価だけ量子」な手法が主流だった

VQE

Variational Quantum Eigensolver

対象: 化学・物質科学

目的: 分子の基底状態エネルギーを求める

例: 水素分子、リチウムイオン電池材料

QAOA

Quantum Approximate Optimization Algorithm

本日

対象: 組合せ最適化

目的: 離散変数のコスト関数の最小化 (近似最適化)

例: MaxCut、TSP、ライツアウト (Sec. 3)

QML

Quantum Machine Learning

対象: 機械学習

目的: パラメータ付き量子回路をモデルとした分類・回帰

例: 量子カーネル法、変分量子分類器

量子シミュレーション

Quantum Simulation

対象: 物理現象の模擬

目的: 量子多体系の時間発展を量子コンピュータ上で再現

例: スピン系、量子磁性、トポロジカル相

VQEに近い手法 (VQS) や時間発展シミュレーションなど
さまざまな手法が登場している

最新 | HPC (富岳) + NISQのハイブリッドアルゴリズムが登場

最適化問題ではなく、NISQでサンプリングデータ作成 → HPCで対角化、という新発想(QSCI)も出てきた

富岳 + IBM Quantum System Two (2025.6~, 神戸)

Heron (156 qubit) と富岳を接続
QSCI (Quantum-Selected Configuration Interaction, Qunasy社提案, [arXiv:2302.11320](https://arxiv.org/abs/2302.11320))を元にした、SQD (Sample-based Quantum Diagonalization, IBM社提案) が運用されている

富岳 + Quantinuum H1「黎明」(2025.2~, 和光・理研)

イオントラップ型・全結合トポロジの黎明を用いて超伝導 (神戸) と異なる物理基盤を用いたアプローチ

電子配置のパターンをNISQで作成し、それを元にHPCで対角化すると系の固有ベクトルがわかる。NISQの利点を活かしたアプローチ

富士通の超伝導 256 qubit 機 (2026.4~, 和光・理研)

国産超伝導量子コンピュータの大規模化
中性原子・イオントラップとは別の国産選択肢として国内 HPC との連携を視野に開発進行中

NVQLink / NVIDIA + 各社 (2025発表)

GPU を古典側として QPU と低遅延接続する規格
Real-time decoder / error mitigation 用途を想定し、量子 SDK ベンダーと多数提携

サンプリングデータの品質はNISQ性能に依存し、(そのため) 現在は超古典的な結果には到達していない
初期のFTQCでの活躍が期待されるアプローチとされる (副読本シリーズで詳述予定)

第1部まとめ：エラー訂正なし → 短い回路 → ハイブリッド路線

近年、NISQに適した（古典に厳しい）計算を使ったハイブリッドアルゴリズムが著しく進化してきた

誤り訂正が現規模では使えない



短い回路で済ます方針を採るしかない



計算全部を量子でなく、難所だけ量子に肩代わりさせる



古典最適化アルゴリズムを分解 → コスト評価だけ量子化



これが VQE / QAOA / QML / 量子シミュレーションの共通形

NISQアルゴリズムの現在地:

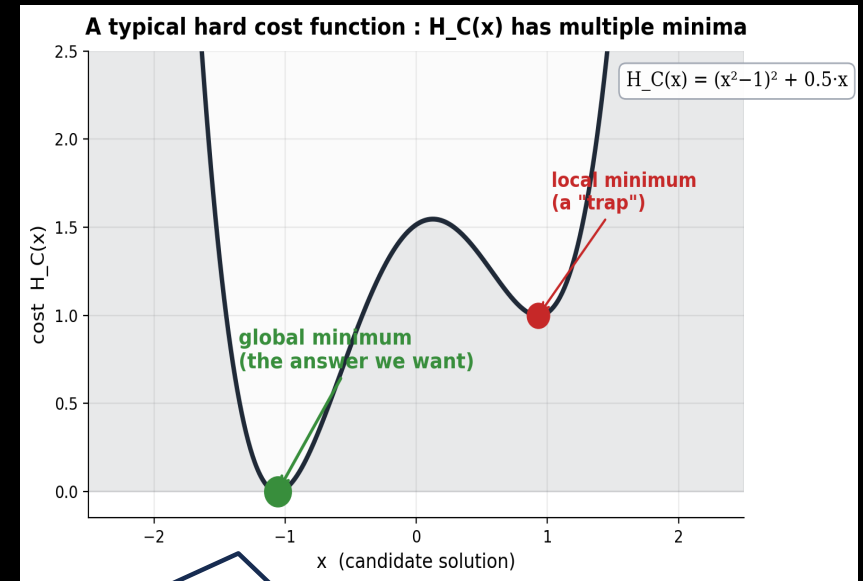
- ・さまざまな提案・報告が登場しているが、古典ヒューリスティクスは依然として強敵
- ・NISQ時代に古典・量子の逆転は困難であっても、初期のFTQCに向けた研究としての重要性は大きい
- ・今日学ぶQAOAは、組合せ最適化で FTQC の有望なサブルーチン候補として注目されている
- ・NISQ での漸近優位の主張はいくつか報告されているが、古典側の改良余地・比較条件の恣意性などの批判があり、確定していない

Sec. 2では QAOA の設計・実装について学んでみよう

Sec. 2

QAOA 入門

量子最適化アルゴリズム (QAA → QAOA)
→ トイモデルの実装 (2 qubit MaxCut)
→ 代数構造 → SAT 系への一般化
(Sec.2 合計 14 枚 / 約 31 分)



最適化問題の例:

- ・この図では大域最適解 (global minimum) は自明だが、変数が多くなると探索領域や局所解は指数的に増加する
- ・古典ではヒューリスティクスを用いて入力の変更と関数の計算を繰り返して探索することになる

1

2

3

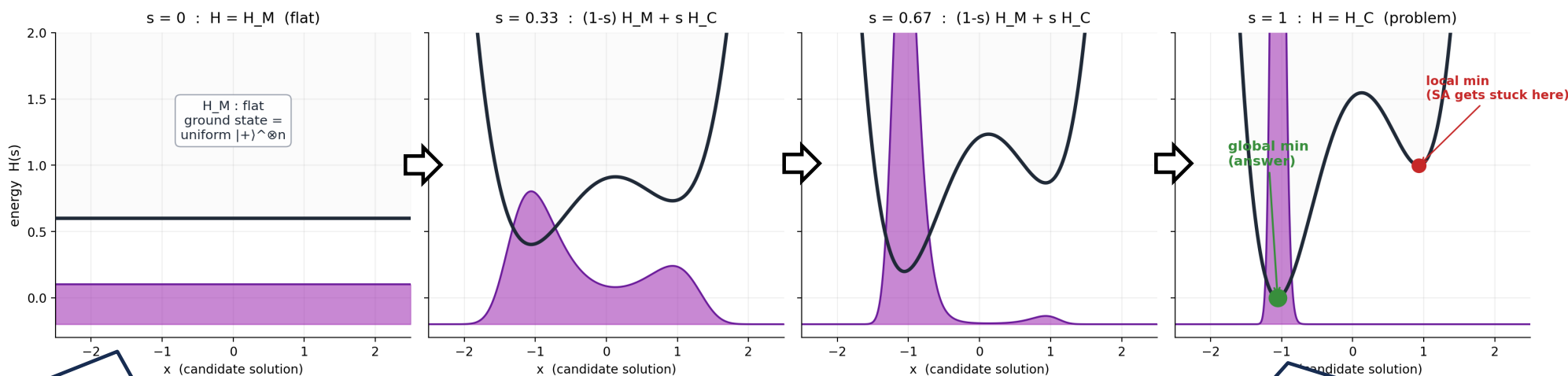
量子断熱計算 (QAA): 実装困難なアルゴリズム

- 時間とともにポテンシャル（黒線）を H_M (平坦：初期状態) $\rightarrow H_C$ (解きたい関数)へと変形させる
- **断熱定理**：波動関数の分布（紫のエリア）は変形に追従し、変形が十分ゆっくりなら最適解が得られる

Quantum Adiabatic Algorithmの略

難問には指数関数的な時間必要

QAA : slowly deform $H_M \rightarrow H_C$, quantum state (purple) keeps total probability = 1



初期状態:
・波動関数は一様分布している

Start uniform on flat $H_M \rightarrow$ landscape gets rugged $\rightarrow$ end concentrated at the global min of H_C
Adiabatic theorem : if T is large enough, the system stays in the ground state at every $s \rightarrow$ ends at global min

終状態:
・波動関数は最適解に集中

QAA を近似したアルゴリズム

収束保証（断熱性）を妥協して、実装可能なアルゴリズムに変形させる

Quantum Approximate
Optimization Algorithmの略

QAA : 連続時間発展 (FTQC なら動く・・・?)

$$H(t) = (1 - s)H_M + sH_C, \quad s = t/T$$

- ・ $t = 0$ で H_M (一様重ね合わせの基底)
- ・ $t = T$ で H_C (求めたい最適解の基底)
- ・ T が大 $\rightarrow$ 大域最小到達確率 高 (断熱定理)



QAOA: “digitization” して実装可能に変形

$$|\gamma, \beta\rangle = \prod_{k=1}^p [e^{-i\beta_k H_M} \cdot e^{-i\gamma_k H_C}] |+\rangle^{\otimes n}$$

- ・ QAAをp stepに分割・近似したアルゴリズム
- ・ pセットのパラメータ (β, γ) を調整する
- ・ ビット列出力の問題に強い

QAOA原著論文: E. Farhi, J. Goldstone, S. Gutmann, "A Quantum Approximate Optimization Algorithm", arXiv:1411.4028 (2014)
(論文・副読本をK-LMSにあげておきました)



QA: 量子アニーリング

- ・ t が物理系依存で調整困難 (断熱性がない)
- ・ D-Wave社が早くから物理実装に成功

富士通のデジタルアニーラなどに派生した
田中宗先生の“量子コンピューティング”で学べます

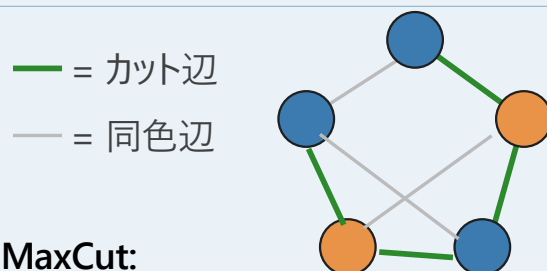
それぞれQAAの精度保証を違う形で失っているため、直接比較は難しい

QAOA: 組合せ最適化を浅い変分回路でサンプリングして解く

古典bit 列を入・出力とし、「浅い量子回路 + optimizer」で「良いコストの x の確率分布」を作り出す

① INPUT (MaxCut問題)

問題を Ising 形 or QUBO で記述する



MaxCut:

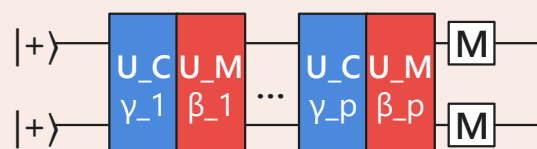
頂点を 2 色に塗り分け、
異色エッジ数を最大化する

Ising 符号化:

$$H_C = \frac{1}{2} \sum (I - Z_i Z_j)$$

② PROCESS

浅い2層構造の繰り返し + optimizer



Cost layer: 良い解に位相をマーク

$$U_C = \exp(-i\gamma H_C)$$

Mixer layer: 干渉で振幅を再分配

$$U_M = \exp(-i\beta H_M), H_M = \sum X_i$$

古典 optimizer: 測定で $\langle H_C \rangle$ 推定
→ (γ, β) 更新 → 再実行

③ OUTPUT

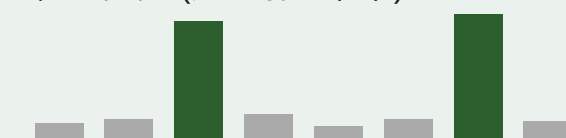
bit 列の確率分布の上位を検証する

β と γ の最適化前 (ほぼ一様分布)



↓ 変分最適化

最適化後 (良い解に集中)



bit 列

最頻 bit 列 = 推定解

なぜ NISQ 向きか: 浅い回路 ($p = 1 \sim 10$ 程度) / 変分パラメータが Trotter 誤差を吸収 / ハードウェアノイズも変分で部分補正

QAOAのトイモデル: 2 qubit MaxCutを解いてみよう

頂点 2, エッジ 1 のグラフで「カットを最大化」する組合せ最適化問題を、まずは古典で定式化する

問題: グラフ $G = (V, E)$



頂点: $V = \{0, 1\}$ エッジ: $E = \{(0, 1)\}$

MaxCut とは: 頂点を 2 色 (青 / 橙) に塗り分け、異色のエッジ数 (カット数) を最大化する問題

解の探索空間: bit 列 $x \in \{0, 1\}^2$

$x = (x_0, x_1)$: 各頂点の色 (0/1)

カット数に対応したCost 関数 (最大化する)

$$C(x) = x_0 + x_1 - 2x_0x_1$$

4 通りの bit 列 → カット数:

$x = 00 \rightarrow$ カット数 0 (同色)

$x = 01 \rightarrow$ カット数 1 (異色 = 最適解 ✓)

$x = 10 \rightarrow$ カット数 1 (異色 = 最適解 ✓)

$x = 11 \rightarrow$ カット数 0 (同色)

量子で定式化するには $|x\rangle = |x_0 x_1\rangle$ として、 $|01\rangle, |10\rangle$ で最大になる Cost Hamiltonian H_C を設計する必要がある

古典の $C(x)$ から量子の H_C を導出する手順を次スライドから具体化する (少し難しいです)

Step 1: bit 表記 $x \in \{0,1\}$ を符号表記 $z \in \{+1,-1\}$ に変換する

Cost関数を Z 演算子 で表現したい $\rightarrow Z$ 演算子の固有値 は ± 1 なので、 x を z で表現する必要がある

天下りの決意表明だが、
コストを位相情報に書き込むために必要な処置

動機: Z 演算子の固有値は ± 1

$Z = [[1, 0], [0, -1]]$ の固有方程式:

$$Z|0\rangle = +1|0\rangle$$

$$Z|1\rangle = -1|1\rangle$$

$\rightarrow$ 固有値の集合は $\{+1, -1\}$

$\rightarrow x \in \{0, 1\}$ のままでは Z で表現できない



対応: Z の固有値に合わせて新変数 z を導入する

変換式:

$$z = 1 - 2x \quad (= (-1)^x)$$

対応表:

$$x = 0 \rightarrow z = +1$$

$$x = 1 \rightarrow z = -1$$

逆変換:

$$x = (1 - z) / 2$$

これで $C(x)$ を Z 演算子の和に置き換える準備が整った

物理ではIsing spin変数と呼ばれる標準的な置換

Next step: 2 qubit MaxCut の Cost 関数 $C(x) = x_0 + x_1 - 2 x_0 x_1$ を z 表記に書き換える

Step 2: Cost 関数 $C(x)$ を Z 演算子の和に書き換える

Step 1で用意した $z = 1-2x$ を $C(x)$ に代入 → さらに z_i を Z_i に置き換えて量子の H_C を導入する

Step 2a: bit 表記の Cost 関数 (古典)

$$C(x) = x_0 + x_1 - 2 x_0 x_1 \quad (\text{異色のとき} = 1, \text{同色のとき} = 0)$$

Step 2b: $z = 1-2x$ を代入し、最終的に Z 演算子の和に変換

z 変換代入: $x_i = (1 - z_i) / 2$ を $C(x)$ に代入して整理
 → $C(z) = (1 - z_0 z_1) / 2$ ($z_0 = z_1$ のとき 0、 $z_0 \neq z_1$ のとき 1)

量子の式に改変する: $z_i \rightarrow Z_i$ (Z 演算子に置き換え)

→ $H_C = (I - Z_0 Z_1) / 2$ ← Cost Hamiltonian 完成

$|01\rangle, |10\rangle$ で H_C の固有値が 1 (最大)、
 $|00\rangle, |11\rangle$ で 0 (最小)となる

Groverはオラクルで正解・不正解の位相反転の有無で表現したように、
 QAOAの H_C はビット列にカット数を連続値で書き込むオラクルとみなせる

Step 3a: 一般グラフのMaxCutへH_Cを拡張する

2 qubit の結果(1 エッジ)を、 $|E|$ 本のエッジを持つ一般グラフへ「エッジで和を取るだけ」で拡張できる

1 エッジの場合 (Step 2までの話)

頂点: $V = \{0, 1\}$ エッジ: $E = \{(0,1)\}$

対象: エッジ1本のみ

$$H_C = (I - Z_0 Z_1) / 2$$

各 $|x\rangle$ の固有値 = カット数 (0 or 1)

ややこしいことを言っているようだが、
H_Cは“カット数を数える対角行列”と考えれば良い

一般グラフの場合: $G = (V, E)$, $|V| = n$, $|E| = m$

各エッジで同じ項を作り、和を取る

$$H_C = \sum_{(i,j) \in E} (I - Z_i Z_j) / 2$$

各 $|x\rangle$ の固有値 = カット数 (整数、0 以上 m 以下)

- ・各項は $Z_i Z_j$ 演算子
- ・項の数 = エッジ数 $|E| = m$

Next step: QAOAの構成とH_Cの役割を考える

Step 3b: QAOA Ansatz

Ansatz: 試行波動関数
最適解を含むかもしれない回路、くらいの意味

QAOAは H_C による時間発展と H_M による混合を交互適用する回路であった。Cost Layerの役割を確認する

QAOA Ansatzの形 (p回繰り返す)

$$|\gamma, \beta\rangle = U_M(\beta_p) \cdot U_C(\gamma_p) \cdot \dots \cdot U_M(\beta_1) \cdot U_C(\gamma_1) \cdot |+\rangle^n$$

- Cost layer: $U_C(\gamma) = \exp(-i\gamma H_C)$
- Mixer layer: $U_M(\beta) = \exp(-i\beta H_M)$
- 初期状態: $|+\rangle^n$ (全ビット列の一樣重ね合わせ)

GroverはOracleで位相反転、Diffusionで増幅した。

QAOAではどれくらい良い解かを連続値で伝えるため

- Cost layerで $C(x)$ に比例した位相を付与し、
- Mixer layerで振幅の再分配を行う。
- 正解が出るかどうかは γ と β の最適化次第となる。

Cost Layerの働き： コストに比例した位相を付与する

$|x\rangle$ は H_C の固有状態 (固有値 $C(x)$) なので:

$$\exp(-i\gamma H_C) |x\rangle = \exp(-i\gamma \cdot C(x)) |x\rangle$$

→ コスト $C(x)$ が大きい $|x\rangle$ ほど位相が大きく回る

この位相差を後段の Mixer が干渉に変え、
解 $|x\rangle$ の確率振幅を増やすのが
QAOA の動作原理

Next step: H_C を量子回路で表現する方法を考える

Step 3c: Cost Layer $\exp(-i\gamma H_C)$ を回路に分解する

H_C はエッジ毎の項の和なので、対応する位相オラクルもエッジ毎のオラクルの積になる

コスト関数には局所性があるため、局所的なゲート操作で置き換えられる

位相オラクル： $\exp(-i\gamma H_C) = \prod_{\{(i,j) \in E\}} \exp(+i\gamma/2 \cdot Z_i Z_j)$

各因子 $\exp(\pm i\theta \cdot Z_i Z_j)$ は 2 qubit $R_{zz}(\theta)$ ゲート 1 つで書ける

カットされる毎に位相 θ を加える
 θ は可変パラメータで
optimizerが最適化していく

Trotter誤差について

一般のハミルトニアンでは A と B は非可換なため
 $\exp(A+B) \approx \exp(A) \cdot \exp(B)$ が近似(Trotter 誤差)となる
MaxCut では全項可換なので厳密に等号が成立する

量子化学計算などでは
大きな問題となるが、
今日は掘り下げない

次は $R_{zz}(\theta)$ ゲートについて考察する

Cost layer の実装 : $R_{zz}(\theta)$ を CX-Rz-CX に分解する

2 qubit ゲート $R_{zz}(\theta) = \exp(-i\theta/2 \cdot Z_i Z_j)$ は NISQ の一般的 native ゲート (CX + 単 qubit) で構成できる

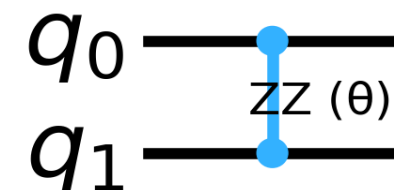
$R_{zz}(\theta)$ is a "parity gate":
even parity $\rightarrow$ phase $-\theta/2$, odd parity $\rightarrow$ phase $+\theta/2$

Input $ ij\rangle$	Phase by $R_{zz}(\theta)$	Parity $i \oplus j$
$ 00\rangle$	$-\theta/2$	even (0)
$ 01\rangle$	$+\theta/2$	odd (1)
$ 10\rangle$	$+\theta/2$	odd (1)
$ 11\rangle$	$-\theta/2$	even (0)

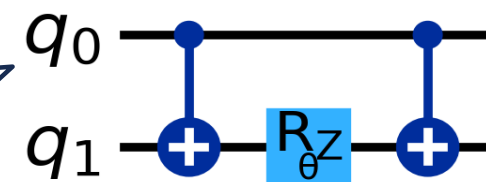
R_{zz} はXORに近い動きをするゲート.
qubitの状態が異なれば位相が増え、
qubitの状態が等しければ位相が減る

CXでXOR値を作って、
Rzで位相 θ を加えて、
CXでuncomputeしている

$R_{zz}(\theta)$ (high-level notation)



= CX - Rz(θ) - CX (NISQ native: 2 CX gates)



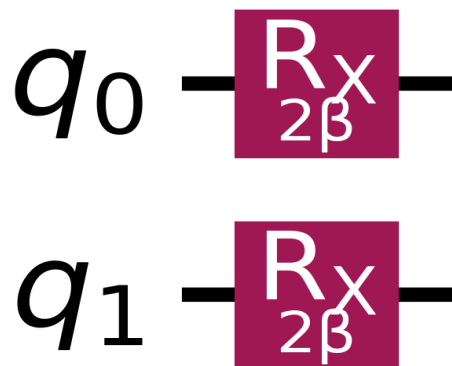
残る謎は Mixer layer $\exp(-i\beta H_M) = H_M = \sum X_i$ は何か? どう回路化するのか?

Mixer layer を実装する

Mixer layer: $\exp(-i\beta H_M) = H_M = \sum X_i$ (一様 X 場) は qubit ごとに独立なので、

全 qubit に $R_x(2\beta) = \text{“X軸まわりに} 2\beta \text{だけ回転させるゲート”}$ を並列適用するだけ

$$\text{Mixer layer } U_M(\beta) = \exp(-i\beta H_M) = \prod_i R_x(2\beta)$$



GroverのDiffusionに相当する操作

- ・パラメータ化されており、QAOA は β で混合強度を連続調整できる
- ・解候補 $|x\rangle$ は Z 基底(計算基底) で記述されている X はその間を行き来させる(部分的ビット反転) ので、振幅を別の $|x\rangle$ に染み出させられる

β = mixing-layer angle: controls how strongly amplitudes "spread" across all candidates

Cost layer と Mixer layer が揃った. これを p 回交互に並べたものが QAOA Ansatzとなる

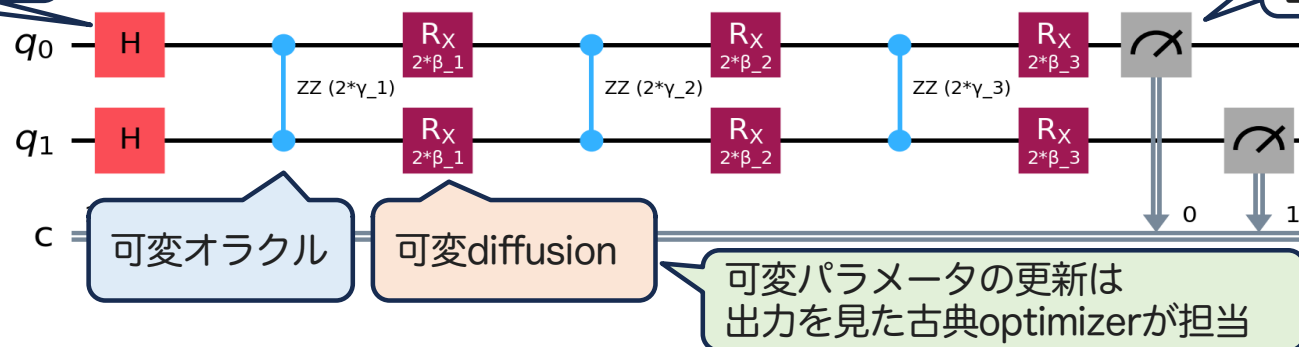
QAOA Ansatz: Cost + Mixer を p 回交互に並べる (本日は p=3)

$|+\rangle^{\otimes n}$ から開始、各層で (γ_k, β_k) パラメータを持つ p layer = $2p$ 個の変分パラメータ

$$|\gamma, \beta\rangle = \prod_{k=1}^p [U_M(\beta_k) \cdot U_C(\gamma_k)] |+\rangle^{\otimes 2}$$

QAOA Ansatz ($n=2, p=3$): 6 variational parameters $\gamma_1, \beta_1, \gamma_2, \beta_2, \gamma_3, \beta_3$

全ての解候補を入力



最適解に到達すると
出力は改善されなくなる

γ_k (Cost-layer angle)

how strongly H_C imprints phase
= "tilt the energy landscape"

β_k (Mixer-layer angle)

how strongly amplitudes are mixed
= "spread the search"

QAOA = Grover の「固定オラクル + 固定 diffusion」を、
「パラメータ可変なオラクル + パラメータ可変な diffusion」に拡張したものと言える

(γ, β) を 1 組固定して $|\psi\rangle$ の変化を追ってみる

手順を理解するための簡易実装 ($p=1$)、実数値は Notebook (Cell A-3) で確認する

演習手順 ($p=1, \gamma=0.5, \beta=0.3$ を例に)

- ① 初期化: $|+\rangle \otimes^2 = (|00\rangle + |01\rangle + |10\rangle + |11\rangle)/2$
→ 各状態の確率は 0.25 で一様
- ② Cost layer: $R_{zz}(2\gamma) = R_{zz}(1.0)$ を適用
→ 各状態に位相が刻まれる
- ③ Mixer layer: $R_x(2\beta) = R_x(0.6)$ を各 qubit に
→ 振幅が混ぜ合わされる
- ④ 測定: 各 $|x_0 x_1\rangle$ の確率を見る
- ⑤ $\langle H_C \rangle = \sum_x p(x) \cdot C(x)$ を計算

期待される性質 (検証ポイント)

- ✓ 解 **$|01\rangle, |10\rangle$** の確率が $|00\rangle, |11\rangle$ より高い
(一様分布 0.25 から偏る方向)
- ✓ $\langle H_C \rangle > 0.5$
(一様な場合は 0.5、それを超えれば最適化が効いている証拠)
- ✓ 最適 $(\gamma^*, \beta^*) \approx (\pi/2, \pi/4)$ 付近で
 $\langle H_C \rangle$ が最大値 **1.0 (= 完全解)** に到達
(2 qubit MaxCut は $p=1$ で完全解可能)

Best な (γ, β) を **自動で探索する** にはどうすればよいだろうか?

COBYLA : 最適化を担う古典の代表的 optimizer

Constrained Optimization BY Linear Approximation: シンプレックス法ベースで勾配不要、関数評価のみで動く

なぜ NISQ では勾配不要な手法が有力なのか

量子側の制約:

- **shot ノイズ** で $\langle H_C \rangle$ の評価値は揺らぐ
- 勾配 $\partial \langle H_C \rangle / \partial \gamma$ を解析的に計算してもノイズの悪影響が大きい
- パラメータシフト則も shot 数が必要

解決策:

- **関数評価のみで進む手法は有力な対策**
- ✓ COBYLA / Nelder-Mead / SPSA など
-

Qiskit / SciPy:

```
scipy.optimize.minimize(method="COBYLA")
```

実際に 2 qubit MaxCut を COBYLA で解いてみよう。何回の評価で $\langle H_C \rangle = 1.0$ に到達するだろうか？

2 qubit MaxCut を QAOA と COBYLA で解く

$p=3$ / 6 パラメータ ($\gamma_1, \beta_1, \gamma_2, \beta_2, \gamma_3, \beta_3$) で実行、関数評価回数と収束を観察

実行設定 (Notebook Cell A-4 で動作確認)

問題: 2 qubit MaxCut Ansatz: $p = 3$ (6 パラメータ) shot 数: 1024 optimizer: COBYLA (maxiter=100)

初期値: ランダム (seed=42) 目的関数: $\langle H_C \rangle$ の最大化 (実装では $-\langle H_C \rangle$ の最小化)

収束の典型的様子

iter	$\langle H_C \rangle$
------	-----------------------

0	0.50	← 一様分布 (初期)
10	0.78	
30	0.95	
50	0.99	

80 ≈ 1.00 ← 完全解到達

最適 (γ^*, β^*) での測定分布

$|00\rangle$: 約 0.00
 $|01\rangle$: 約 0.50 ← 解
 $|10\rangle$: 約 0.50 ← 解
 $|11\rangle$: 約 0.00

解候補 $|01\rangle, |10\rangle$ の合計確率 ≈ 1.0
→ 2 qubit MaxCut は完全解可能

Sec. 3

3×3 ライツアウトに QAOA を適用、Grover と比較する

XOR-SAT 帰着 → 多体 H_C → 9 qubit Ansatz → COBYLA 実演 → Grover と CX 数で比較 を 11 枚 で辿る (Sec.3 合計 12 枚 / 約 30 分)

1

2

3

Brute force と QAOA の比較 : 古典の全探索 vs 量子の確率的探索

9 qubit なら $2^9 = 512$ 通りの全探索が瞬時にできる。なぜ QAOA を使うのか？

古典 brute forceによる 3x3ライツアウト攻略

- ・候補数: $2^9 = 512$
- ・各候補で $A \cdot x \oplus b$ を評価 $O(n^2)$
- ・総計算量: $2^9 \times 81 \approx 41000$ 演算
- ・実行時間: マイクロ秒

一般 $n \times n$ ライツアウトの場合

- ・候補数 $2^{(n^2)} \rightarrow$ 指数爆発
- ・ $n=10$ なら $2^{100} \approx 10^{30}$ で brute force 不可
- ・GF(2) Gauss 消去なら $O(n^6)$ で解ける
→ 線形構造なら 古典が依然優位

QAOA + ライツアウト (これから設計する)

- ・ $138 \text{ CX} \times \text{shots} \times \text{maxiter} \approx 10^5 - 10^6 \text{ ops}$
- ・解確率 $\sim 5-16\%$ 、top-3 の検算を経て解が確定する

QAOAは役に立たない・・・？

- ・線形構造を持たない問題 (一般 MaxCut)
ではQAOAも有効になる (Sec.2 Slide 13)
- ・ライツアウト $\equiv$ XOR-SAT は古典 Gauss で解けるが、Cost layer の構造を観察できる良教材なので採用した
- ・3x3 ライツアウトで QAOA の振る舞いを掴み、
より複雑な問題に応用する基礎にしてください

私たちは中間課題で既にライツアウトを Grover で解いている。どちらが NISQ で実用的なのだろうか？

ライツアウト：全消灯を目指す近傍反転パズル

中間レポートではGroverで解いたばかりですが、今日はQAOAで解いてみよう

問題ルール

- ・ $3 \times 3 = 9$ マスのライトは {消灯 or 点灯}
- ・ マス i を押すと:
 - ・ i 自身 + 上下左右の近傍のライトが **反転 (XOR)**

目標: 初期盤面から全消灯に至る
ボタン操作の組合せ x を見つける

操作変数: $x = (x_0, x_1, \dots, x_8) \in \{0, 1\}^9$
($x_i = 1 \Leftrightarrow$ ボタン i を押す)

盤面の例 (初期状態 b)

初期盤面
 $b = (1, 1, 1, 0, 0, 0, 1, 0, 1)$

黄色: 点灯 ($b_i = 1$)
灰色: 消灯 ($b_i = 0$)

全消灯にするには・・・?

0	1	2
3	4	5
6	7	8

解くための準備: ボタン操作と盤面の関係を **線形代数で表現する** ▶ $A \cdot x = b \pmod{2}$ ▶ XOR-SAT 帰着

XOR-SAT 帰着 : ライツアウトを GF(2) 上の線形方程式に変換する

ボタン操作の効果は近傍反転 = XOR、これを行列 A で表現すれば $A \cdot x = b \pmod{2}$

隣接行列 A (9×9 , GF(2))

$A_{\{i,j\}} = 1 \Leftrightarrow$ ボタン j がマス i を反転

具体的には:

- $i = j$ (自分) のとき $A_{\{i,i\}} = 1$
- i, j が 4-近傍 (上下左右) のとき 1
- それ以外は 0

3x3 の場合:

- A は 正則 ($\det A = 1 \pmod{2}$)
- 任意の b に対し解 x が一意に存在

XOR-SAT 形式 (GF(2) 上の方程式)

解くべき方程式:

$$A \cdot x \equiv b \pmod{2}$$

- $x \in \{0,1\}^9$: 押すボタンの組合せ
- $b \in \{0,1\}^9$: 初期盤面 (点灯パターン)
- 各方程式 = 1 つのライトの "消灯条件"

点いている数をカウントする:

$$C(x) = \|A \cdot x - b\|^2 \text{ (制約 violation 数)}$$

→ 最小値 0 (= 完全消灯) を達成する x が解となる

具体的な b と解 x を見ながら、Cost 関数 $C(x)$ を Z 演算子化してみよう

多体 Cost 関数の構築：XOR 制約を Z 演算子の積で表現する

Sec.2 の Step 2 と同じ手順を 9 qubit + 多体項に拡張する

Step 1: Cost 関数 (bit 表記、各制約の violation 数の和)

$$C(x) = \sum_{i=0}^8 [(\sum_j A_{\{i,j\}} x_j) \oplus b_i] \quad \leftarrow \text{各マスの "消灯条件" の違反 (XOR) を合計}$$

各項は マス i の近傍数 (3, 4, 5) に応じた多変数 XOR になる

Step 2: bit $\rightarrow$ z 変換 ($z_i = 1 - 2x_i$) $\rightarrow$ 多体 Z 演算子の和に

$$H_C = \sum_{i=0}^8 (I - (-1)^{b_i} \cdot \prod_{j: A_{\{i,j\}}=1} Z_j) / 2$$

- 各項は 近傍数 = 3, 4, 5 個の Z 演算子の積 (= 多体項)
- b_i の符号がそのまま位相に乗る (+1 or -1)
- 全項は Z 同士の積なので 全可換 $\rightarrow$ Trotter 誤差 0 (Maxcut と同じ性質)

多体項の内訳 (3-body, 4-body, 5-body) は具体的に何項ずつあるだろうか?

注: k -body 項 = k 個の qubit にまたがる項 (例: 3-body = 3 qubit に Z 演算子が並ぶ $Z_i Z_j Z_k$)

H_Cの構造 : 3-body × 4 + 4-body × 4 + 5-body × 1 = 9 項

盤面のトポロジー (角 / 辺 / 中央) がそのまま多体次数の内訳を決める

近傍数 × マス数 = 項数

マス位置	近傍数	項数
角 (0, 2, 6, 8)	3	× 4 = 4 個の 3-body 項
辺 (1, 3, 5, 7)	4	× 4 = 4 個の 4-body 項
中央 (4)	5	× 1 = 1 個の 5-body 項

合計 = 9 項

これが 9 qubit ライツアウト H_C の全構成

2 qubit MaxCut の 2-body よりも次数が高い
→ Cost layer の回路が深くなる

盤面位置 ↔ 多体次数 の対応

0 3-body	1 4-body	2 3-body
3 4-body	4 5-body	5 4-body
6 3-body	7 4-body	8 3-body

■ 3-body ■ 4-body ■ 5-body

次は、多体 Z 演算子の指数 $\exp(-i\gamma Z_{\{i1\}} \cdots Z_{\{ik\}})$ をどう回路化するか考えよう

参考 | 多体 Cost layer 実装 : CX chain で $\exp(-i\theta Z \otimes Z \otimes \dots \otimes Z)$ を構成

2 qubit の Rzz の自然な拡張 = CX chain という手法が知られている

k-body Z 演算子

$$\exp(-i\theta \cdot Z_{\{i_1\}} Z_{\{i_2\}} \dots Z_{\{i_k\}})$$

3 stepでの実装法 (CX chain, 右図) :

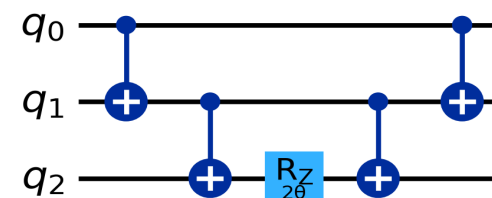
① $CX_{\{i_1, i_2\}}, CX_{\{i_2, i_3\}}, \dots, CX_{\{i_{k-1}, i_k\}}$
で位相を最後の qubit に集約

② $Rz_{\{i_k\}}(2\theta)$ で位相を刻む

③ ステップ ① を逆順に適用して戻す

3-body 例: $\exp(-i\theta Z_0 Z_1 Z_2)$

$$\exp(-i\theta Z_0 Z_1 Z_2) = \text{CX chain} - Rz(2\theta) - \text{CX chain} \quad (\text{CX count} = 4)$$



Qiskit では:
`PauliEvolutionGate(H_C, time=y)`
 で全 9 項を自動構成 (transpile で CX 数最適化)することも出来る

9 個の項 (3-body $\times 4$ + 4-body $\times 4$ + 5-body $\times 1$) に対して QAOA回路全体 を組むと CX は何個になるだろうか？

9 qubit Ansatz 完成 : $p=3$ で総 CX 数 138 個

Cost layer 1層あたりの CX 数 (transpile + optimization_level=1 後)

項	CX 数 = $2(k-1)$	項数	小計
3-body ($k=3$)	4	$\times 4$	= 16
4-body ($k=4$)	6	$\times 4$	= 24
5-body ($k=5$)	8	$\times 1$	= 8

Cost layer 1層 の合計 = 48 CX

transpile optimization 後 $\approx$ 46 CX

Ansatz 総計 ($p=3$) と回路 depth

- Cost layer $\times p = 46 \times 3 = 138$ CX
- Mixer layer $\times p = 9 \times 3 = 27$ Rx (CXは0)
- 初期化 = 9 H (CXは0)

CX 総数 = 138 個

CX chainは 1次元接続に親和性があるため、
オーバーヘッドが発生していない

COBYLA で 6 パラメータ ($\gamma_1, \beta_1, \gamma_2, \beta_2, \gamma_3, \beta_3$) を最適化して、本当に解 $|010010101\rangle$ が出るか試してみよう

9 qubit QAOA + COBYLA で実行してみよう

本日の主実験: $p=3$ / $b=(1,1,1,0,0,0,1,0,1)$ で解 $(1,0,1,0,1,0,0,1,0)$ が top-3 内に出るだろうか？

実行設定 (Notebook Cell C-3)

- ・ 問題: 3×3 ライツアウト (9 qubit)
- ・ $b = (1,1,1,0,0,0,1,0,1)$
- ・ 真の解 $x = (1,0,1,0,1,0,0,1,0)$
- ・ 目的関数: $\langle H_C \rangle$ の最小化
(最小固有値 = 0 = 完全解)

参考 | notebookで採用したパラメータ

- ・ Ansatz: $p = 3$ (6 パラメータ)
- ・ optimizer: COBYLA
- ・ shots: 5000-10000
- ・ maxiter: 100-200, rhobeg: 0.5
- ・ 初期値: 一様乱数 $[0, 2\pi]$

評価指標

- ・ 真の解 $|010010101\rangle$ が top-3 累積で 5% 以上
→ PASS とする (合格基準)

なぜ top-3 か:

- ・ QAOA は確率的サンプリング
- ・ 上位候補を 古典的に検算 する運用が現実的
(各候補 x について $A \cdot x \oplus b = 0$ を mod 2 で確認)

参考 | 論文を書くときに気になる評価指標

- ・ $\langle H_C \rangle_{\min}$ の収束値
- ・ 関数評価回数
- ・ シード依存性

実行して (3分くらいかかります)、top 候補に真の解が含まれるか確認しよう

実行結果

どの seed でも $p=3$ で top-3 に解が得られた！

結果 (5 seed)

seed	$\langle H_C \rangle_{\min}$	解の確率	解の rank
42	≈ 3.3	1.3 %	rank 2
1	≈ 3.1	1.7 %	rank 1
7	≈ 3.5	4.7 %	rank 2
100	≈ 3.2	1.5 %	rank 2
2024	≈ 3.2	1.2 %	rank 2

検証済: 真の解 $|010010101\rangle$ は 全 5 seed で top-3 内に出現 (累積 ~5-16%)

Cell C-5 | 最適な繰り返し回数 p_{opt}

p	parameters, #CX	解確率
1	2, 46	低い
2	4, 92	~3-7%
3 (採用)	6, 138	~5-16%

- p が大きいほど 解確率は上昇するが回路が深くなる
- ノイズを考慮すると p を上げて改善しないことがある
- $p=3$ は 理論精度と実装可能性の妥協点
- 最適な p は Goemans-Williamson 比で得られる (副読本)

観察と運用上の意味

- 単一実行の解確率は ~5-16% と低いが、top-3 を古典的に検算する運用なら確実に解に到達
- NISQ アルゴリズムは「量子で候補を狭めて古典で検算する」戦略が基本となる (Sec.1 Slide 5-7)

最後に、QAOA と Grover の特性を比較してみよう

QAOA と Grover を比較してみよう

GroverのデータはIBM Qiskit Challenge ([Link](#)) を参照した

手法	CX 数 (p=3, k iter)	解の確率	事前知識要件	特徴
QAOA (本日) reps = 3	138	~5-16% (シード依存)	不要 (汎用)	確率的 要古典
Grover k = 4 (同確率比較)	624	~16%	解の数 M (または推定)	確定的 振幅増幅
Grover k_opt = 17 (最大)	2652	~100%	解の数 M 精密に必要	$k_{\text{opt}} = \pi/4 \cdot \sqrt{(N/M)}$
Grover + 前処理 k_opt = 2	221	~92%	問題構造を精査	$k_{\text{opt}} = \pi/4 \cdot \sqrt{(N/M)}$

観察と運用上の意味

- ・ QAOA 成功確率はシード・初期値依存。また、古典で検算する必要がある。
- ・ Grover は k_{opt} を知るために解の数 M が事前に必要になる。
- ・ NISQ で一般問題を解く場合には、QAOA の軽さ・汎用性が利点になるだろう。

Sec.3 まとめ : QAOA と Grover の使い分け方

QAOAを学び、同じライツアウトを 2 つの量子アルゴリズムで解いた

QAOA が向く問題

- ・ 解の数 M が不明
- ・ 組合せ最適化 (MaxCut, TSP, 配送計画)
- ・ 近似解で十分 (厳密解不要)
- ・ 古典 optimizer と組合せられる
- ・ NISQ 雑音に対し **相対的に頑健** (変分で調整)

欠点:

- ・ 成功確率が低い (~5-16%)
- ・ barren plateau (qubit 数増加で勾配消失)
- ・ 収束保証は限定的 (実証的アプローチ)

Grover が向く問題

- ・ 解の数 M が既知 or 推定可能
- ・ SAT、データベース検索、暗号解読
- ・ 厳密解が必要
- ・ oracle を効率的に組める
- ・ **確定的に $\sqrt{N}$ 倍の加速** (古典 $O(N) \rightarrow O(\sqrt{N})$)

欠点:

- ・ 解の数 M を知らないと k_{opt} が決まらない
- ・ oracle 構築が問題ごとに必要
- ・ CXゲートが相対的に多く、NISQ での運用は難しい

結論: 実問題では **組合せ最適化の需要が多い + まだNISQ時代** → **QAOA 系統が現在の主流**

次回 #7: 量子機械学習 同じハイブリッド戦略 (変分パラメータ + 古典 optimizer) を分類タスクへ応用

副読本送り？：講義本編で扱わなかった論点

NISQ#2 副読本 §10 で扱う論点を整理中・・・

数学的・理論的論点

- §10.1 QAOA の収束保証 ($p \rightarrow \infty$)
- §10.2 Trotter 誤差と p 選択の理論
- §10.3 Grover との詳細比較 (CX 数の精密測定)
- §10.4 XOR-SAT の古典最適 (Gauss 消去 $O(n^6)$)
- §10.5 Goemans-Williamson 近似比 0.7071
- §10.6 Adiabatic 定理と QAA からの導出
- §10.7 barren plateau (勾配消失問題)
- §10.8 warm start (古典近似解で QAOA を初期化)
- §10.9 Mixer の拡張 (XY mixer、constraint preserving)
- §10.10 ZX-calculus による回路最適化
- §10.11 Pauli string の可換クラス分解

実装・運用論点

- §10.12 実機 NISQ (Heron, Brisbane) での実行
- §10.13 noise model と error mitigation
- §10.14 SPSA optimizer (確率的勾配近似)
- §10.15 parameter-shift rule
- §10.16 INTERP / FOURIER 初期化戦略
- §10.17 一般 $n \times n$ ライツアウト (5×5 , 7×7 拡張)
- §10.18 QAOA とアニーリングの関係
- §10.19 量子機械学習との接続 (QML)
- §10.20 FTQC 時代の QAOA の位置付け
- §10.21 古典シミュレーション可能性の境界
- §10.22 Qiskit Runtime / Aer サンプリングの詳細

本日の講義は **QAOA の動作原理 + 実装 + Grover との比較** に集中。深い理論は副読本で。

解答例 1 / 2 : Part A ~ B (TODO 1 ~ 4)

Notebook の TODO を埋めるための解答コード (講義後の自己学習用、時間余裕時のみ言及)

TODO 1 (A-3) : QAOA Ansatz の Cost + Mixer (4 行)

```
qc.rzz(2 * gammas[k], 0, 1)
qc.rx(2 * betas[k], 0)
qc.rx(2 * betas[k], 1)
```

TODO 2 (B-2) : 隣接行列の 2 箇所 (2 行)

- ① A[idx, idx] = 1
- ② A[idx, 3*nr+nc] = 1

TODO 3 (B-4) : 多体 Pauli の Z 配置 (1 行)

```
for j in range(n):
    if A[i, j] == 1: p[j] = "Z"
```

TODO 4 (B-6) : 9 qubit Ansatz の Cost + Mixer (3 行)

```
qc.append(PauliEvolutionGate(H_C, time=gammas[k]), range(n_qubits))
for i in range(n_qubits):
    qc.rx(2 * betas[k], i)
```

解答例 2 / 2 : Part C (TODO 5 ~ 6)

Notebook の TODO を埋めるための解答コード (講義後の自己学習用、時間余裕時のみ言及)

TODO 5 (C-2) : COBYLA で最適化を実行 (2 行)

```
result = minimize(cobyla_objective, init, method="COBYLA",  
                  options={"maxiter": 100, "rhobeg": 0.5})
```

TODO 6 (C-3) : 古典検算 verify_candidate (3 行)

```
x = np.array([int(c) for c in bitstr_qiskit[::-1]])  
residual = (A @ x + b) % 2  
return x, residual, (residual == 0).all()
```